

Multiplierless FIR Filter Design with Parallel-Cut Ellipsoid Method

Wai-Shing Luk

Abstract—A general-purpose multiplier is the most expensive arithmetic element in a digital finite impulse response (FIR) filter, so a filter whose coefficients are represented in canonical signed digit (CSD) form can be realized with a small number of adders and shifts instead of multipliers. The design problem, however, is awkward: the magnitude constraints are not convex in the impulse response, the coefficient quantizer is not convex at all, and the constraint set is effectively infinite because the frequency response must be bounded at every frequency. This article shows that all three difficulties are handled naturally by the ellipsoid method. Spectral factorization turns the magnitude specifications into convex constraints on the autocorrelation, the two-sided magnitude bounds are returned by the oracle as parallel cuts that shrink the search ellipsoid faster than a single cut, and the CSD quantizer is embedded in the oracle so that the optimizer reasons about a realizable coefficient vector rather than a relaxed one. The complete pipeline, from a frequency-domain specification to a synthesizable shift-and-add description, is described and implemented in three languages. Experiments quantify the iteration economy of the parallel-cut formulation — a reduction of 77 to 97 percent — and compare the running time across the three languages over four filter orders, and the generated RTL is synthesized to report technology-independent cell, lookup-table, and flip-flop counts.

I. INTRODUCTION

A finite impulse response (FIR) filter computes a weighted sum of the recent input samples,

$$y[t] = \sum_{k=0}^{n-1} h[k] u[t-k],$$

and is specified by its frequency response $H(\omega) = \sum_{m=0}^{n-1} h(m)e^{-jm\omega}$. Its design consists of choosing the coefficients $h[0], \dots, h[n-1]$ so that $|H(\omega)|$ stays inside a prescribed mask. The classical tools for this task — windowing and frequency sampling [27], the Parks–McClellan exchange algorithm [29], METEOR [34], and peak-constrained least-squares design [1, 32] — all assume that a full multiplier is available for each coefficient.

In application-specific integrated circuits and field-programmable gate arrays the multiplier is precisely the element one wants to avoid: it dominates area and power, and it becomes disproportionately expensive as the word length grows. A *multiplierless* filter replaces each product by a signed sum of shifted copies of the input. Because a shift by a power of two is free in dedicated hardware,

a coefficient that is a sum of a few signed powers of two can be realized with a few adders and no multiplier [26, 30, 31]. The design problem is therefore not merely to find good coefficients, but to find coefficients that are *cheap to realize*.

The design of filters whose coefficients are confined to a discrete set has a long history, and the multiplierless case is its most hardware-driven instance. The finite-wordlength problem — choosing coefficients from a fixed grid rather than from the continuum — was among the earliest digital-filter design questions, and the first systematic attacks cast it as an integer program over a discrete coefficient space [16, 22]. Because enumeration grows exponentially with the number of taps, the literature turned to branch-and-bound [15, 21] and to local search seeded from a continuous relaxation [20]. Restricting the coefficients to powers of two, or to a signed sum of a few powers of two, proved especially attractive in hardware, and specialized search procedures were developed for it [23, 24, 31]. Later work exploited structure in the coefficient space: a trellis formulation solved the signed-power-of-two problem by dynamic programming [6], metaheuristics such as genetic algorithms were applied where the search space resisted exact methods [11, 37], and lattice reduction was used to obtain optimal finite-wordlength designs [19]. A parallel line of work characterized the performance achievable under a word-length constraint [17, 18]. These methods are exact, or nearly so, only under assumptions that are difficult to satisfy in practice: integer programming becomes prohibitive as the order grows, and the heuristics return a feasible design only when the search happens to fall into a feasible basin.

A second line of work keeps the coefficients continuous and reshapes the problem so that convex optimization applies. Spectral factorization turns the magnitude specification into a linear constraint on the autocorrelation [12, 36], and the resulting convex program is solved by second-order cone or semidefinite methods, with the spectral mask encoded as a linear matrix inequality [8, 35] and implemented in disciplined-convex-programming tools such as CVX [13]. The survey of [9] collects many such reformulations. Because these methods return a continuous optimum, however, they answer a different question from the one the hardware asks. The optimum typically lies on the boundary of the feasible set, where the active magnitude constraints are tight, so rounding it to the nearest CSD vector can push part of the response below its lower bound and destroy feasibility [9, 20]. A relaxation can supply a lower bound

or a warm start, but it cannot by itself certify a realizable design.

The ellipsoid method has meanwhile been applied well beyond linear programming, and it is the natural algorithm whenever the constraint set is convex but its description is implicit or infinite. Prior applications of the same principle include robust analog circuit sizing with affine arithmetic [25], parametric network optimization through optimal matrix scaling [28] and minimum mean-cycle computation [7], and multi-parameter clock-skew scheduling [39]. What these applications share, and what multiplierless FIR design inherits, is a moderate number of design variables combined with a very large number of constraints — the regime in which the ellipsoid method is competitive with interior-point methods [4, 5]. The distinguishing feature of the present article is that it keeps the quantizer inside the oracle, so that the optimizer reasons about a realizable coefficient vector rather than a relaxed one.

Two obstacles stand in the way. First, the magnitude constraint

$$L(\omega) \leq |H(\omega)| \leq U(\omega), \quad \forall \omega \in [0, \pi], \quad (1)$$

is not convex in the coefficient vector $\mathbf{h}$; and it must hold at every frequency, so it is a semi-infinite constraint. Second, the set of coefficients representable with a bounded number of signed powers of two is discrete, and the natural procedure of optimizing first and rounding afterwards can destroy feasibility. This article addresses both by treating the whole design as a feasibility problem for the ellipsoid method, whose only requirement is a separation oracle.

The article is organized as follows. §II reformulates the magnitude specifications as convex constraints on the autocorrelation via spectral factorization. §III recalls the ellipsoid method and develops the parallel-cut update in the form used throughout. §IV introduces the canonical signed digit representation and the associated adder cost. §V embeds the quantizer in the oracle. §VI compares two spectral-factorization algorithms. §VII describes the implementation and reports cross-language results, and §VIII concludes.

II. DERIVING A CONVEX DESIGN PROBLEM

Work with the squared magnitude rather than the magnitude. The *autocorrelation* of the impulse response is

$$r[k] = \sum_i h[i] h[i+k], \quad k = 0, 1, \dots, n-1, \quad (2)$$

and the squared magnitude is its cosine series

$$R(\omega) = |H(\omega)|^2 = r[0] + 2 \sum_{k=1}^{n-1} r[k] \cos(k\omega) = \mathbf{a}(\omega)^\top \mathbf{r}, \quad (3)$$

where $\mathbf{a}(\omega) = (1, 2 \cos \omega, \dots, 2 \cos((n-1)\omega))^\top$. The map $\mathbf{r} \mapsto R(\omega)$ is affine, so the *squared magnitude bounds*

$$L^2(\omega) \leq R(\omega) \leq U^2(\omega), \quad R(\omega) \geq 0, \quad \forall \omega \in [0, \pi], \quad (4)$$

define a convex set in $\mathbf{r}$. This is the classical spectral-decomposition device of [36], building on the spectral factorization of [12]; the same convexification is surveyed in [9].

The problem solved in this article is

$$\begin{aligned} & \text{minimize} && \gamma, \\ & \text{subject to} && L^2(\omega) \leq R(\omega) \leq U^2(\omega), \quad \omega \in \Omega_p, \\ & && R(\omega) \leq \gamma, \quad \omega \in \Omega_s, \\ & && R(\omega) \geq 0, \quad \omega \in [0, \pi], \end{aligned} \quad (5)$$

where Ω_p and Ω_s are the passband and stopband, and γ bounds the stopband energy. The optimization variable is the autocorrelation $\mathbf{r}$; the frequency response is recovered afterwards by spectral factorization (§VI). Because the constraints must hold on a continuum, and because the number of design variables is moderate while the number of constraints is large, this is exactly the regime in which the ellipsoid method is competitive with interior-point methods [4, 5].

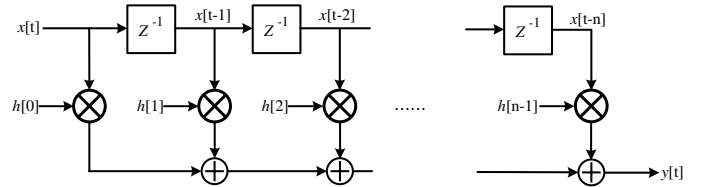


Figure 1: A typical structure of an FIR filter.

III. THE PARALLEL-CUT ELLIPSOID METHOD

A. Separation Oracles and Cutting Planes

Let $\mathcal{K} \subset \mathbb{R}^n$ be compact and convex. A *separation oracle* Ω , queried at a point x_0 , either certifies that $x_0 \in \mathcal{K}$ or returns a hyperplane that separates x_0 from $\mathcal{K}$:

$$g^\top(x - x_0) + \beta \leq 0, \quad \beta \geq 0, \quad g \neq 0, \quad \forall x \in \mathcal{K}. \quad (6)$$

The pair (g, β) is a *cutting plane*: it discards the half-space where $g^\top(x - x_0) + \beta > 0$. If $\beta = 0$ the cut is *central*, if $\beta > 0$ it is *deep*, and if $\beta < 0$ it is a *shadow cut*. When $\mathcal{K}$ is defined by $f_j(x) \leq 0$, a cut is obtained for free from a subgradient, $(g, \beta) = (\partial f_j(x_0), f_j(x_0))$; for a differentiable f_j the subgradient is the gradient. The method was introduced independently by Shor [33] and by Yudin and Nemirovskii [38], and used by Khachiyan to prove that linear programming is polynomial-time solvable [14]; see the survey of [2, 3].

B. The Ellipsoid Update

The search space is the ellipsoid

$$\mathcal{E} = \{x \mid (x - x_c)^\top Q^{-1}(x - x_c) \leq \kappa\}, \quad (7)$$

with center x_c , shape matrix $Q \succ 0$, and scale $\kappa > 0$. Writing

$$\tilde{g} = Qg, \quad \omega = g^\top \tilde{g}, \quad \tau = \sqrt{\kappa \omega},$$

every cut type applies the same update

$$x_c^+ = x_c - \frac{\rho}{\omega} \tilde{g}, \quad Q^+ = Q - \frac{\sigma}{\omega} \tilde{g} \tilde{g}^\top, \quad \kappa^+ = \delta \kappa, \quad (8)$$

and differs only in the scalar triple (ρ, σ, δ) . For a central cut ($\beta = 0$),

$$\rho = \frac{\tau}{n+1}, \quad \sigma = \frac{2}{n+1}, \quad \delta = \frac{n^2}{n^2-1}, \quad (9)$$

and for a deep cut with offset $\beta > 0$,

$$\begin{aligned} \rho &= \frac{\tau + n\beta}{n+1}, \\ \sigma &= \frac{2(\tau + n\beta)}{(n+1)(\tau + \beta)}, \\ \delta &= \frac{n^2}{n^2-1} \cdot \frac{\tau^2 - \beta^2}{\tau^2}. \end{aligned} \quad (10)$$

The application of the update to Q alone, rather than to a general positive definite matrix, saves n^2 floating-point operations per iteration; the split into κ and Q is what makes the scalar τ cheap to compute [3].

C. Parallel Cuts

The magnitude specifications (Eq. 1) are two-sided, and this is where the method pays off. Whenever a constraint has the form

$$l \leq a^\top x + b \leq u,$$

the oracle can return a *pair* of parallel cuts that share a normal g :

$$\begin{aligned} g^\top(x - x_c) + \beta_0 &\leq 0, \\ g^\top(x - x_c) + \beta_1 &\geq 0, \quad \forall x \in \mathcal{K}. \end{aligned} \quad (11)$$

The pair removes a slab from the ellipsoid rather than a half-space, which shrinks the volume more than either cut alone and therefore reduces the number of iterations [10]. A single deep cut is the special case in which one of the two planes is tangent.

The geometry is worth stating explicitly. Parameterize the current ellipsoid along the cut normal by $s = g^\top(x - x_c)/\tau$, so that s ranges over $[-1, 1]$ on $\mathcal{E}$; the two cuts retain only the band $-\beta_1/\tau \leq s \leq -\beta_0/\tau$. The complement of the band consists of two caps, one where s is too large and one where it is too small. A single cutting plane removes only one cap, whereas the pair removes both in the same iteration, so the least-volume ellipsoid that must contain the survivors is tighter. For a magnitude specification the two caps have a physical meaning: they are the frequency regions where the response exceeds the upper bound and falls below the lower bound, and the parallel cut discards both violations at once. Two degenerate cases anchor the formulas below. When the band is centered, $\beta_0 + \beta_1 = 0$, the new center coincides with the old one and the update must remain finite. When one plane is tangent, $\beta_1 = \tau$, the band collapses to a half-space and the update must reduce to the deep cut Eq. 10.

The auxiliary quantities have direct interpretations. Each $\zeta_i = \tau^2 - \beta_i^2$ is the room that remains along g beyond plane i : it is positive exactly when that plane cuts the ellipsoid ($|\beta_i| \leq \tau$) and vanishes when the plane is tangent. The combination $\eta = \tau^2 + n\beta_0\beta_1$ is the admissibility

test, because $\eta > 0$ is equivalent to $\beta_0\beta_1 > -\tau^2/n$, the condition under which a strictly smaller enclosing ellipsoid exists; the case $\eta \leq 0$ is precisely the “no smaller ellipsoid” branch quoted below. The term ξ is the positive root of the quadratic that remains after the two tangency conditions are combined; it is the quantity that couples the two planes, and it collapses to the single-cut value when they coalesce. Finally, the displacement $\rho = \sigma(\beta_0 + \beta_1)/2$ moves the center toward the middle of the band, which is why a centered band leaves the center fixed.

The update itself is obtained by choosing the least-volume ellipsoid that contains the intersection of $\mathcal{E}$ with the band. Equivalently, it minimizes $\ln \det Q^+$ over the ellipsoids that contain the band, and the optimum is characterized by both planes being supporting hyperplanes of the new ellipsoid. The stationarity conditions force the rank-one form Eq. 8 with two scalar multipliers; imposing tangency at both planes eliminates one multiplier and leaves a quadratic whose positive root is ξ , which yields Eq. 13 and Eq. 14. The central and deep cuts are the limiting tangency cases, and the complete algebra is worked out in [10].

The complete update uses the auxiliary quantities

$$\begin{aligned} \zeta_0 &= \tau^2 - \beta_0^2, \\ \zeta_1 &= \tau^2 - \beta_1^2, \\ \xi &= \sqrt{\zeta_0\zeta_1 + \left(\frac{n}{2}(\beta_1^2 - \beta_0^2)\right)^2}, \end{aligned} \quad (12)$$

and, with $\eta = \tau^2 + n\beta_0\beta_1$,

$$\sigma = \frac{2\eta}{\tau^2 + \beta_0\beta_1 + \frac{n}{2}(\beta_0 + \beta_1)^2 + \xi}, \quad \rho = \sigma \cdot \frac{\beta_0 + \beta_1}{2}, \quad (13)$$

$$\delta = \frac{n^2}{(n^2-1)\tau^2} \left(\frac{\zeta_0 + \zeta_1}{2} + \frac{\xi}{n} \right). \quad (14)$$

This formulation is finite even when the two offsets are symmetric, $\beta_0 + \beta_1 = 0$, which is precisely the case that defeats a mean-offset expression dividing by $(\beta_0 + \beta_1)^2$. It also degenerates correctly: setting $\beta_1 = \tau$ makes the second plane tangent and reduces the parameters to the deep-cut values Eq. 10 with $\beta = \beta_0$. The intersection is empty when $\beta_0 > \beta_1$; no smaller ellipsoid exists when $\beta_0\beta_1 < -\tau^2/n$; and the update reduces to a deep cut when $\beta_1^2 > \tau^2$.

D. Volume Reduction

Applying the matrix-determinant lemma to Eq. 8,

$$\begin{aligned} \det Q^+ &= (1 - \sigma) \det Q, \\ \frac{\text{vol}(\mathcal{E}^+)}{\text{vol}(\mathcal{E})} &= \delta^{n/2} (1 - \sigma)^{1/2} \leq e^{-1/(2n)}. \end{aligned} \quad (15)$$

After k iterations the volume is at most $e^{-k/(2n)}$ of the initial volume, so reaching a fraction ϵ requires $k \approx 2n \ln(1/\epsilon)$ iterations. The determinant of Q therefore decreases monotonically, and the factor δ stays in $(0, n^2/(n^2-1))$.

A robust implementation stores a factorization $Q = \kappa LDL^\top$ with L unit lower triangular and D diagonal, and

Algorithm 1 Cutting-plane optimization with parallel cuts

Require: oracle Ω , ellipsoid $\mathcal{E} \supseteq \mathcal{K}$, tolerance ϵ

Ensure: value γ^* , coefficients $\mathbf{h}_{\text{csd}}$

```

1:  $\gamma \leftarrow +\infty$ 
2: repeat
3:    $x_c \leftarrow \text{center}(\mathcal{E})$ 
4:    $(g, \beta_0, \beta_1, t) \leftarrow \Omega(x_c, \gamma)$ 
5:   if  $t < \gamma$  then
6:      $\gamma \leftarrow t$ ;  $\mathcal{E} \leftarrow \text{CENTRALCUT}(\mathcal{E}, g, 0)$ 
7:   else if  $\beta_1^2 \leq \tau^2$  then
8:      $\mathcal{E} \leftarrow \text{PARALLELCUT}(\mathcal{E}, g, \beta_0, \beta_1)$ 
9:   else
10:     $\mathcal{E} \leftarrow \text{DEEPCUT}(\mathcal{E}, g, \beta_0)$ 
11:   end if
12: until  $\text{vol}(\mathcal{E}) < \epsilon$ 
13: return  $\gamma^*, \mathbf{h}_{\text{csd}}$ 

```

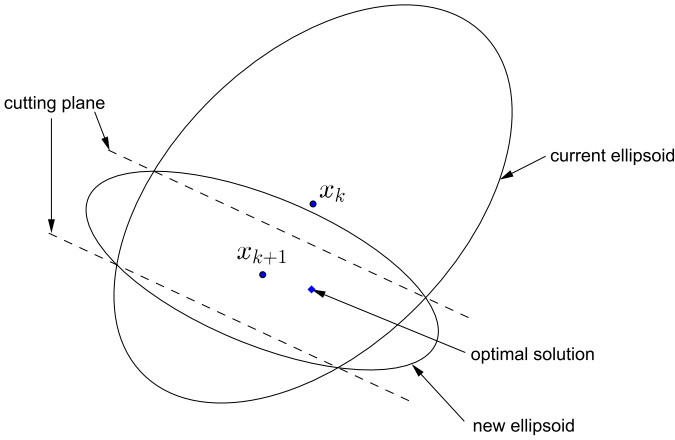


Figure 2: Parallel cuts.

applies the rank-one update to (L, D) directly. Positive definiteness is then preserved by construction, which prevents the silent loss of definiteness that repeated rank-one downdates can cause in the unfactored form.

IV. CANONICAL SIGNED DIGIT AND ADDER COST

A coefficient is realized without a multiplier when it is a signed sum of powers of two,

$$c = \sum_j s_j 2^{e_j}, \quad s_j \in \{-1, 0, +1\}, \quad (16)$$

because each term is a shift and each pair of terms is an add or subtract. A canonical signed digit (CSD) representation is one in which no two adjacent digits are non-zero; it is unique and contains the fewest non-zero digits of any signed-digit representation of the same number [30]. For example,

$$28.5 = +00-00.+0_{\text{csd}} = 32 - 4 + 0.5,$$

which uses three non-zero digits rather than the four set bits of the binary form. A power of two, such as $h_i = 2^{-3}$, needs no adder at all.

Algorithm 2 CSD quantization with a non-zero-digit budget

Require: value x , budget nnz

Ensure: c with at most nnz non-zero digits

```

1:  $c \leftarrow 0$ ;  $b \leftarrow 2^{\lceil \log_2(1.5|x|) \rceil - 1}$ 
2: while  $\text{nnz} > 0$  and  $|x| > 0$  do
3:   if  $|1.5x| > b$  then
4:      $c \leftarrow c + \text{sign}(x)b$ 
5:      $x \leftarrow x - \text{sign}(x)b$ ;  $\text{nnz} \leftarrow \text{nnz} - 1$ 
6:   end if
7:    $b \leftarrow b/2$ 
8: end while
9: return  $c$ 

```

The number of adders per coefficient is one less than the number of non-zero digits, before any sharing. Because several coefficients are realized simultaneously, the relevant problem is *multiple constant multiplication* (MCM): the goal is to build a network of adders and shifts that realizes all coefficients at minimum cost. *Common subexpression elimination* (CSE) searches for bit patterns shared across coefficients and reuses their computation, reducing the adder count and, importantly, the *adder depth* — the number of adder stages, which controls the critical path [26, 31]. The number of non-zero digits is therefore a design parameter, not an accident of the coefficient values.

To expose the trade-off, a coefficient is quantized to the nearest CSD number with at most a prescribed number of non-zero digits, written

$$h_{\text{csd}}[k] = \text{csd}(h[k], \text{nnz}). \quad (17)$$

Enlarging the budget nnz improves the approximation and enlarges the set of feasible filters, at the price of more adders; shrinking it to one or two digits makes many otherwise realizable filters infeasible. The budget is a constraint of the design problem, and it is what makes the problem discrete.

A. Shift-Add Synthesis and Common Subexpressions

A CSD coefficient is realized by a shift for every non-zero digit and an add or subtract for every subsequent digit, so a coefficient with d non-zero digits costs $d - 1$ adders before any sharing. Two forms of sharing reduce this cost. Within a coefficient, a signed pattern that occurs at positions p and $q > p$ satisfies

$$\text{pat}_q(x) = \text{pat}_p(x) \gg (q - p),$$

so one shift-and-add network serves both occurrences, and the longest repeated substring of the digit string identifies the pattern worth sharing. Across coefficients, the same idea selects the pattern that maximizes

$$\text{score} = (\text{nnz} - 1)(\text{occurrences} - 1),$$

which is exactly the number of adders saved by sharing that pattern. The percentage saved is a property of the

pattern, not of the filter: how much of the saving a whole design realizes depends on how many patterns are common across coefficients. The reference generator of §VII shares one dominant cross-coefficient pattern, so the whole-design reduction it achieves is modest, and a search over all patterns would remove more.

B. Filter Architecture

The same coefficient set admits a direct or a transposed realization. The direct form forms each product $h[k]x[n-k]$ and accumulates them; the transposed form pipelines the accumulator and applies the coefficients in reverse order, producing one output with a single-cycle latency. Both forms use the same CSD strings and therefore the same per-tap hardware, but they differ in register count, adder depth, and external wiring. The transposed form is the natural default for a self-contained module, whereas the direct form exposes the individual products for a custom accumulator tree or a polyphase decomposition.

V. QUANTIZATION-AWARE DESIGN

Designing a multiplierless filter is not a matter of optimizing first and rounding afterwards. At the relaxed optimum the active magnitude constraints are saturated, so the optimum lies on the boundary of the feasible set; the composite map $S^{-1} \circ \text{csd} \circ S$, which rounds a coefficient vector and returns its autocorrelation, is neither a projection nor exact, and even a small perturbation can push part of the passband below its lower bound. The rounded design can therefore be substantially infeasible.

The remedy is to move the quantizer inside the oracle. The pipeline is

$$\mathbf{r} \longrightarrow \mathbf{h} = S(\mathbf{r}) \longrightarrow \mathbf{h}_{\text{csd}} \longrightarrow \mathbf{r}_{\text{csd}} = S^{-1}(\mathbf{h}_{\text{csd}}), \quad (18)$$

where S is spectral factorization and S^{-1} recomputes the autocorrelation of the quantized response. The oracle is queried at a continuous center $\mathbf{r}$ and proceeds as follows.

- **Quantize.** Compute the continuous factor $\mathbf{h} = S(\mathbf{r})$ and replace every coefficient by its nearest CSD value under the budget: $h_{\text{csd}}[k] = \text{csd}(h[k], \text{nnz})$.
- **Re-evaluate exactly.** Recompute $\mathbf{r}_{\text{csd}} = S^{-1}(\mathbf{h}_{\text{csd}})$ and test the sampled constraints against $\mathbf{r}_{\text{csd}}$, not against $\mathbf{r}$. A feasible point then corresponds to a genuinely realizable coefficient vector. Because the cut is generated at the quantized point rather than at the queried center, the oracle re-anchors it with the first-order correction $\beta \leftarrow \beta + g^T(\mathbf{r}_{\text{csd}} - \mathbf{r})$, so that the cut is valid at $\mathbf{r}$ and can be applied directly.
- **Classify.** The candidate is feasible (improve the best-so-far value and take a central cut), violated (take a deep cut from the gradient at $\mathbf{r}_{\text{csd}}$), or ineffective (the cut does not shrink the ellipsoid).
- **Retry.** The ineffective case is peculiar to the discrete setting: because the admissible set is finite, several consecutive centers can map to the same CSD pattern, producing a cut that removes no new volume. The

Algorithm 3 Quantization-aware oracle Ω_Q

Require: center $\mathbf{r}$, budget nnz , retry flag, count t , grid size m

Ensure: a cut (g, β) , possibly an improved γ , and whether a retry is allowed

- 1: **if** not retry **and** $\mathbf{r} \notin \mathcal{K}$ **then**
- 2: **return** cut at $\mathbf{r}$
- 3: **end if**
- 4: $\mathbf{h} \leftarrow \text{SPECTRALFACT}(\mathbf{r})$
- 5: $\mathbf{h}_{\text{csd}} \leftarrow \text{CSDQUANTIZE}(\mathbf{h}, \text{nnz})$
- 6: $\mathbf{r}_{\text{csd}} \leftarrow \text{INVERSESPECTRALFACT}(\mathbf{h}_{\text{csd}})$
- 7: **if** $\mathbf{r}_{\text{csd}} \in \mathcal{K}$ **then**
- 8: $\gamma \leftarrow f_0(\mathbf{r}_{\text{csd}})$
- 9: **return** $(\partial f_0(\mathbf{r}_{\text{csd}}), 0)$ ▷ central cut
- 10: **else**
- 11: $g \leftarrow \partial f_j(\mathbf{r}_{\text{csd}})$; $\beta \leftarrow f_j(\mathbf{r}_{\text{csd}}) + g^T(\mathbf{r}_{\text{csd}} - \mathbf{r})$
- 12: **return** (g, β) , retry allowed iff $t < m$ ▷ deep cut, or retry
- 13: **end if**

oracle replies with a retry flag and a budget equal to the number of sampled frequency rows, $m = c_{\text{disc}}n$; each retry advances a round-robin cursor over the constraints, so that a different violated row is examined instead of the one that produced the ineffective cut. When the budget is exhausted the oracle returns the last cut even if it is shallow, which lets the ellipsoid update proceed; termination is then governed by the usual volume test. Bounding the retries by the grid size guarantees that each oracle call makes progress or reports its inability to do so, so the outer loop cannot stall on a single discrete pattern.

One pitfall deserves emphasis. The oracle certifies feasibility only on the finite grid of $m = c_{\text{disc}}n$ sampled frequencies. Between samples the squared magnitude can dip below its lower bound even though the design passes at every sample; this is a *discretization artifact*, the familiar consequence of replacing a semi-infinite constraint by finitely many samples. It is mitigated by enlarging the sampling factor c_{disc} when the constraints are tight, and by re-checking the final design on a finer grid. The fine-grid check is advisory, because the optimizer never certified that resolution.

VI. SPECTRAL FACTORIZATION

Both the convex formulation and the oracle require a spectral factorization: given the autocorrelation Eq. 2, find the minimum-phase impulse response h that produces it. The minimum-phase factor is unique when r corresponds to a positive spectrum, and it is the factor a causal implementation stores.

Transform-based method (Kolmogorov). Evaluate the power spectrum $R(\omega)$ on an oversampled grid, form the log-magnitude $\alpha(\omega) = \frac{1}{2} \ln R(\omega)$, obtain the minimum-phase phase as its Hilbert transform $\phi = \mathcal{H}[\alpha]$, and return

$$h = \mathcal{F}^{-1}[\exp(\alpha(\omega) + j\phi(\omega))]. \quad (19)$$

The method is non-iterative, deterministic, and stable, and it has no tuning parameter; its costs are an FFT dependency and a memory footprint proportional to the oversampling factor, which is customarily 100 times the filter order.

Root-based method (Aberth–Ehrlich). The autocorrelation defines the palindromic polynomial

$$P(z) = z^{N-1} \left(r[0] + \sum_{k=1}^{N-1} r[k] (z^k + z^{-k}) \right)$$

of degree $2N - 2$, whose roots occur in reciprocal pairs: if ζ is a root then $1/\bar{\zeta}$ is also a root. All roots are found simultaneously by the Aberth–Ehrlich iteration

$$\zeta_i^{(k+1)} = \zeta_i^{(k)} - \frac{P(\zeta_i^{(k)})}{P'(\zeta_i^{(k)})} / \left(1 - \sum_{j \neq i} \frac{P(\zeta_j^{(k)})}{(\zeta_i^{(k)} - \zeta_j^{(k)}) P'(\zeta_j^{(k)})} \right),$$

the roots inside the unit circle are retained, and the factor is reconstructed from them. The method needs no FFT library, uses memory linear in the order, and exposes a convergence tolerance; convergence is not guaranteed for pathological inputs, and the smallest coefficients are recovered with somewhat lower relative accuracy.

Reconstruction from roots is sensitive to the order in which the linear factors are multiplied. Multiplying well-separated roots first — a Leja ordering — keeps the intermediate coefficients from growing, and reduces the reconstruction error from roughly 10^{-2} to machine precision. A second numerical point is the convergence test: an absolute residual $|P(z)|$ is not scale-free, and when the coefficients are large the threshold it implies can lie below the attainable floating-point floor, so the iteration never reports convergence. Scaling the polynomial by $s = \max(1, \max_k |a_k|)$ leaves the iteration invariant — both the correction and its denominator scale the same way — while making the tolerance relative, which restores convergence without changing the roots.

The natural measure of a factorization is its round-trip relative error, the agreement between the prescribed r and the autocorrelation of the returned factor. The transform method attains close to machine precision, on the order of 10^{-5} in double precision for representative designs, whereas the root-based method is around 10^{-3} , concentrated in the smallest coefficients. Because CSD quantization discards precisely those coefficients once the budget is applied, the discrepancy is usually absorbed by the quantizer, but it must be accounted for in verification.

Which method is faster is a property of the implementation rather than of the algorithm. When the root finder is compiled and the FFT is not, the root-based method can be the faster choice; when both are compiled, the two methods are comparable and the crossover depends on the order. This observation motivates the cross-language experiments of §VII. The transform method is the default for production designs, for high-order filters, and for

Algorithm 4 Multiplierless FIR design pipeline

Require: specification $(n, \omega_p, \omega_s, L, U)$, budget nnz

Ensure: a realizable coefficient vector $\mathbf{h}_{\text{csd}}$

- 1: initialize $\mathcal{E}$ and $\gamma \leftarrow +\infty$
 - 2: **repeat**
 - 3: $\mathbf{r} \leftarrow \text{center}(\mathcal{E})$
 - 4: $(\mathbf{h}_{\text{csd}}, \text{cut}) \leftarrow \Omega_Q(\mathbf{r}, \text{nnz})$
 - 5: $\mathcal{E} \leftarrow \text{UPDATE}(\mathcal{E}, \text{cut})$
 - 6: **until** $\text{vol}(\mathcal{E}) < \epsilon$
 - 7: **return** $\mathbf{h}_{\text{csd}}$
-

reproducible benchmarks, because it needs no tuning and is the most robust; the root-based method is preferable for exploratory work and for moderate orders, where its speed and tunability dominate.

VII. IMPLEMENTATION AND EXPERIMENTS

A. The Software Pipeline

The pipeline of Eq. 18 has been implemented three times, in Python, Rust, and C++, from a single algorithmic specification. The heavy components are shared: an ellipsoid engine supplies the search space and the generic cutting-plane loop; a spectral-factorization routine supplies S ; and a CSD quantizer supplies $\text{csd}(\cdot, \text{nnz})$. The oracle checks the passband, stopband, and non-negativity constraints in round-robin order and returns the first violation it finds, which preserves the round-robin cursor that prevents the same cut from being reintroduced. The implementations are checked by property-based testing: a fast implementation is compared against a reference on randomly generated inputs, which exposes silent conversion and factorization defects that example-based unit tests miss.

B. Experimental Setup

The reference design is a lowpass filter with passband edge 0.12π , stopband edge 0.20π , a CSD budget of $\text{nnz} = 7$ non-zero digits, a discretization factor $c_{\text{disc}} = 15$, a tolerance of 10^{-14} , and an initial ellipsoid radius of 40. Four orders are used, $n \in \{16, 32, 64, 128\}$, which range from a small design to one whose dimension makes the $O(n^2)$ iteration budget visible. All three implementations read the same specification and share the same defaults, so any difference in the results is attributable to the language and its standard libraries rather than to the algorithm. Measurements were taken on a Windows x64 machine with release builds of the three implementations; each figure is the mean of five measured runs taken after two warm-up runs.

C. Cross-Language Results

Two observations stand out. First, the compiled implementations are an order of magnitude or more faster than the interpreted one at small orders — about thirty-two times at $n = 16$ and ten times at $n = 32$ — but the gap narrows as the order grows, to about 2.9 times at $n = 128$.

Table I: Running time and iteration count by filter order, averaged over five measured runs. The three implementations share one specification; the figures are indicative wall-clock times from the authors’ implementations.

Order n	Mean time (ms)			Iterations		
	Python	C++	Rust	Python	C++	Rust
16	1977	61	50	620	625	625
32	2786	268	252	1931	1850	2530
64	4224	997	1041	1555	1585	1558
128	6713	2322	2303	1784	1869	1842

The narrowing is informative: the interpreted cost is the dispatch of the Python-level loop over the constraints, which is amortized against an ever-larger amount of FFT work as the order rises, whereas the compiled cost is dominated by the arithmetic itself. The two compiled implementations are within ten percent of each other at every order, and neither dominates consistently: C++ is faster at $n = 32$ and 64, Rust at $n = 16$ and 128.

Second, the iteration counts differ across languages even though the inputs are identical, and they are not monotone in the order. The ellipsoid method is a first-order method with no mechanism for converging to a unique optimum, so floating-point differences in the FFT libraries and in the accumulation order lead to different, equally valid trajectories; the solutions satisfy the same specifications and have comparable energy $\sum h[k]^2$. The counts also show that the number of iterations is not a proxy for the order: $n = 32$ requires more iterations than $n = 64$ in every language, because the parallel-cut geometry, not the dimension, determines how much volume each cut removes.

D. Iteration Economy of Parallel Cuts

To isolate the effect of the parallel-cut update, the same designs were run with the parallel-cut path disabled, so that the oracle could remove only one half-space per iteration.

The parallel cut reduces the iteration count by between 77 and 97 percent, and the gap widens with the order: at $n = 64$ and $n = 128$ the single-cut variant exhausts the 50,000-iteration budget without reaching the tolerance, whereas the parallel-cut variant converges in fewer than 1,900 iterations. This is the quantitative form of the observation of §III: the two-sided magnitude constraints are what the parallel update is for, and a single-cut method pays for the lower bound and the upper bound in separate iterations.

E. Post-Synthesis Hardware Cost

The emitted shift-and-add RTL was synthesized to confirm that the design is realizable in hardware and to give a technology-independent cost. Two flows were run in Yosys: a generic gate-level flow, which maps the datapath to elementary logic and flip-flops and reports a target-independent cell count, and the Xilinx flow, which maps the same RTL to six-input lookup tables and flip-flops.

The design contains no multiplier or DSP cells, and the cost is dominated by the carry chains of the adders and by the pipeline registers: the flip-flop count is essentially n

times the output word length. Both the lookup-table and the generic cell counts grow a little faster than linearly in the order, because the word length itself grows with n . The $n = 128$ design was not carried through the synthesis flow, which did not complete within a practical time budget on the available machine; the trend across the three smaller orders is consistent.

These figures should be read as a synthesis-level estimate, not as a power or area measurement. No place-and-route was performed, no dynamic power was measured, and the counts are specific to the mapping used; the software running times of the three implementations are a proxy for algorithmic cost and carry no direct hardware meaning. A power comparison against a multiplier-based realization would require a full synthesis, place-and-route, and simulation flow, which is outside the scope of this article.

F. Profiling

Profiling the interpreted implementation produced a counter-intuitive result: the spectral factorization accounted for only about six percent of the running time, while the constraint scan accounted for about eighty-three percent. The scan issued one short dot product per constraint — on the order of half a million tiny calls — and the interpreter dispatch, not the arithmetic, dominated. Replacing the per-constraint dot products with a single matrix-vector product followed by boolean masks, while preserving the round-robin cursor, made the scan primitive about 94 times faster and the end-to-end run about 1.6 times faster. After the change the run was dominated by library import time.

The same change did not transfer to C++. Porting the vectorization literally made the compiled implementation about 1.9 times *slower*, because the original compile-time per-row loop stops at the first violation and therefore performs about half the work of a full matrix-vector product. The lesson is that a vectorization win observed in an interpreted language must be re-measured after compilation; the measurement, not the conclusion, is what ports. Exposing raw pointers to the compiler and using a short chain of independent partial sums (rather than a single scalar reduction, which the compiler may not reassociate) recovered a further small margin in the compiled implementations.

Table II: Iteration count with a single cut and with parallel cuts. The single-cut variant did not reach the tolerance within the 50,000-iteration budget at the two largest orders.

Order n	Single cut	Parallel cut	Reduction
16	2666	625	76.6%
32	19,038	1850	90.3%
64	$\geq 50,000$	1585	$\geq 96.8\%$
128	$\geq 50,000$	1869	$\geq 96.3\%$

Table III: Post-synthesis cost of the emitted RTL for the transposed cross-CSE design. The lookup-table and flip-flop counts come from a six-input-LUT mapping; the generic cell count is target-independent. The design uses no multiplier or DSP cells.

Order n	Lookup tables	Flip-flops	Generic cells
16	4850	704	27,115
32	10,661	1472	57,074
64	22,946	3264	130,225

G. Optimization and Sharing Results

The same kernel changes paid off very differently across the three languages:

The CSD coefficient set is compressed further by common-subexpression sharing:

The per-pattern figures above are the savings from a single shared pattern, not from a whole design. For $\text{nnz} = 7$ the flat count is six adders per tap — 96 adders at $n = 16$ and 768 at $n = 128$ — and after the cross-pattern sharing performed by the reference generator the counts are 91 and 745, respectively. Closing the remaining gap requires a multiple-constant-multiplication search over all patterns, which is left to future work (§VIII).

H. Multiplierless Result

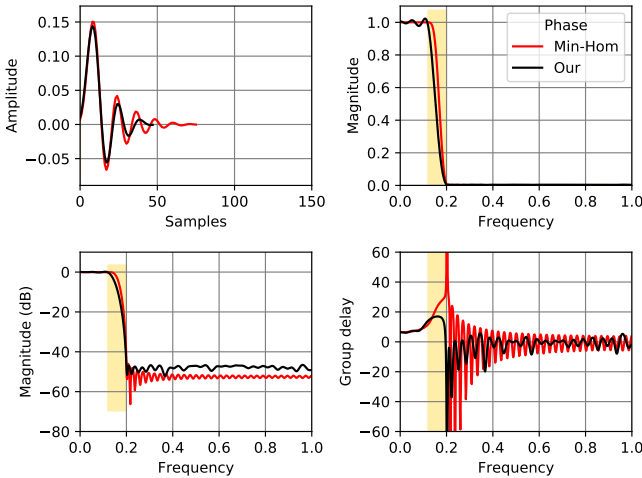


Figure 3: Magnitude response of a lowpass design.

The output of the pipeline is the quantized coefficient vector $\mathbf{h}_{\text{csd}}$ together with its adder network. Because the quantizer was consulted during the optimization rather than after it, the returned design satisfies the sampled specifications by construction, and the only remaining verification is to confirm that the design also holds between the samples of the grid. The shift-and-add description is then emitted

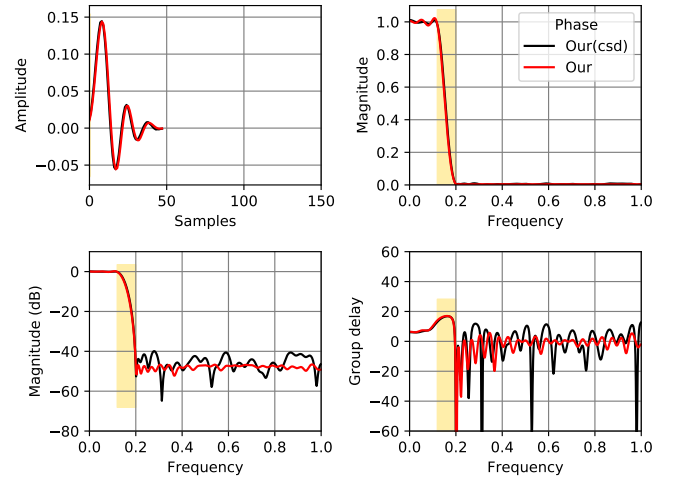


Figure 4: Magnitude response after CSD quantization.

directly for synthesis, and the post-synthesis cell and register counts of the preceding subsection confirm that the result is realizable in hardware without a multiplier.

VIII. CONCLUDING REMARKS

The ellipsoid method is an unusually good fit for multiplierless FIR design. Its separation oracle accommodates the semi-infinite magnitude constraints without ever enumerating them; the two-sided nature of those constraints is returned as parallel cuts, which reduce the iteration count; and the discrete CSD quantizer can be embedded in the oracle so that the optimizer searches over realizable coefficient vectors rather than over a relaxation. The same formulation applies unchanged to the robust and parametric variants of the design problem, and the numerical experiments confirm that a compiled implementation of the whole pipeline runs in under 2.4 seconds for orders up to 128, that the parallel-cut update keeps the iteration count below 1,900 where a single-cut method does not converge, and that the emitted RTL synthesizes to logic and registers without any multiplier cells.

Three directions remain open. The first is a sharper analysis of the parallel cut under a discretization grid, so that the

Table IV: Effect of the kernel changes in three languages.

Change	Language	Before	After	Speedup
Vectorized constraint scan	Python	3.05 s	0.082 s	37×
End-to-end after the scan	Python	3.01 s	1.87 s	1.6×
Raw-pointer dot product	C++	0.172 s	0.160 s	1.07×
Four-accumulator dot	Rust	23.3 ns	10.1 ns	2.3×
Oracle scan	Rust	13.9 μ s	10.7 μ s	1.3×

Table V: Adder count with and without common-subexpression sharing.

CSD pattern	Flat adders	Shared adders	Saving
+0-0+0-0	4	2	50%
+0-0+0-0+0-0	6	2	67%
+00-00+00-00	4	2	50%

fine-grid advisory check can be replaced by a guarantee. The second is to fold the adder-sharing stage — the multiple-constant-multiplication network that realizes the CSD coefficients — into the oracle, so that adder cost, and not only coefficient error, is a first-class design objective. The third is a systematic study of the filter architecture: the direct and transposed forms realize the same transfer function but differ substantially in adder depth and register count, and the choice interacts with the coefficient ordering that the CSD quantizer produces; the synthesis counts reported in §VII are a first datapoint, not a substitute for that study.

IX. ACKNOWLEDGMENTS

The author is grateful to the contributors of the open-source packages on which this work depends.

REFERENCES

- [1] Adams, J. W., and Sullivan, J. L., Peak-constrained least-squares optimization, *Signal Processing, IEEE Transactions on* **46** (1998) 306–321.
- [2] Bland, R. G., Goldfarb, D., and Todd, M. J., The ellipsoid method: A survey, *Operations Research* **29** no. 6 (1981) 1039–1091.
- [3] Bland, Robert G, Goldfarb, Donald, and Todd, Michael J, The ellipsoid method: A survey, *Operations research* **29** no. 6 (1981) 1039–1091.
- [4] Boyd, S, and Barratt, C, Ellipsoid method, *Lecture Slides on Class EE364b of Stanford University, Available: <http://www.stanford.edu/class/ee364b/lectures.html>* (2008).
- [5] Boyd, Stephen, and Vandenberghe, Lieven, *Convex optimization*, Cambridge university press, 2009.
- [6] Chen, Chao-Liang, and Willson Jr, Alan N, A trellis search algorithm for the design of FIR filters with signed-powers-of-two coefficients, *Circuits and Systems II: Analog and Digital Signal Processing, IEEE Transactions on* **46** no. 1 (1999) 29–39.
- [7] Dasdan, Ali, and Gupta, Rajesh K, Faster maximum and minimum mean cycle algorithms for system-performance analysis, *IEEE transactions on computer-aided design of integrated circuits and systems* **17** no. 10 (1998) 889–899.
- [8] Davidson, T. N., Luo, Z.-Q., and Sturm, J. F., Linear matrix inequality formulation of spectral mask constraints with applications to FIR filter design, *Signal Processing, IEEE Transactions on* **50** (2002) 2702–2715.
- [9] Davidson, Timothy N, Enriching the art of FIR filter design via convex optimization, *Signal Processing Magazine, IEEE* **27** no. 3 (2010) 89–101.
- [10] Frenk, Johannes Bartholomeus Gerardus, Gromicho, Joaquim, and Zhang, Shuzhong, A deep cut ellipsoid algorithm for convex programming: Theory and applications, *Mathematical programming* **63** nos. 1-3 (1994) 83–108.
- [11] Glover, Fred, and Kochenberger, Gary A, *Handbook of metaheuristics*, Springer, 2003.
- [12] Goodman, T. N. T., Micchelli, C. A., Rodriguez, G., and S. Seatzu, and, Spectral factorization of laurent polynomials, *Advances Comput. Math.* **7** no. 4 (1997) 429–454.
- [13] Grant, Michael, Boyd, Stephen, and Ye, Yinyu, CVX: Matlab software for disciplined convex programming, 2008.
- [14] Khachiyan, L. G., A polynomial algorithm in linear programming, *Soviet Mathematics Doklady* **20** (1979) 191–194.
- [15] Kodek, Dusan, An algorithm for the design of optimal finite word-length FIR digital filters, in *Acoustics, speech, and signal processing, IEEE international conference on ICASSP'80.*, IEEE, pp. 73–76.
- [16] Kodek, Dusan M, Design of optimal finite wordlength FIR digital filters using integer programming techniques, *Acoustics, Speech and Signal Processing, IEEE Transactions on* **28** no. 3 (1980) 304–308.
- [17] Kodek, Dusan M, Limits of finite wordlength FIR digital filter design, in *Acoustics, speech, and signal processing, 1997. ICASSP-97., 1997 IEEE international conference on*, IEEE, pp. 2149–2152.
- [18] Kodek, Dusan M, Performance limit of finite wordlength FIR digital filters, *Signal Processing, IEEE Transactions on* **53** no. 7 (2005) 2462–2469.
- [19] Kodek, Dusan M, LLL algorithm and the optimal finite wordlength FIR design, *Signal Processing, IEEE Transactions on* **60** no. 3 (2012) 1493–1498.

- [20] Kodek, Dusan, and Steiglitz, Kenneth, Comparison of optimal and local search methods for designing finite wordlength FIR digital filters, *Circuits and Systems, IEEE Transactions on* **28** no. 1 (1981) 28–32.
- [21] Lawler, Eugene L, and Wood, David E, Branch-and-bound methods: A survey, *Operations research* **14** no. 4 (1966) 699–719.
- [22] Lim, Yong Ching, Parker, S, and Constantinides, AG, Finite word length FIR filter design using integer programming over a discrete coefficient space, *Acoustics, Speech and Signal Processing, IEEE Transactions on* **30** no. 4 (1982) 661–664.
- [23] Lim, Yong Ching, Yang, Rui, Li, Dongning, and Song, Jianjian, Signed power-of-two term allocation scheme for the design of digital filters, *Circuits and Systems II: Analog and Digital Signal Processing, IEEE Transactions on* **46** no. 5 (1999) 577–584.
- [24] Lim, Yong, and Parker, S, FIR filter design over a discrete powers-of-two coefficient space, *Acoustics, Speech and Signal Processing, IEEE Transactions on* **31** no. 3 (1983) 583–591.
- [25] Liu, Xuexin, Luk, Wai-Shing, Song, Yu, Tang, Pushan, and Zeng, Xuan, Robust analog circuit sizing using ellipsoid method and affine arithmetic, in *Proceedings of the 2007 asia and south pacific design automation conference*, IEEE Computer Society, pp. 203–208.
- [26] Marković, Dejan, and Brodersen, Robert W, *DSP architecture design essentials*, Springer, 2012.
- [27] Oppenheim, Alan V, Schafer, Ronald W, Buck, John R, and others, *Discrete-time signal processing*, vol. 2, Prentice hall Englewood Cliffs, NJ, 1989.
- [28] Orlin, James B, and Rothblum, Uriel G, Computing optimal scalings by parametric network algorithms, *Mathematical programming* **32** no. 1 (1985) 1–10.
- [29] Parks, T. W., and McClellan, J. H., Chebyshev approximation for nonrecursive digital filters with linear phase, *Circuit Theory, IEEE Transactions on* **CT-19** no. 2 (1972) 189–194.
- [30] Reitwiesner, George W., Binary Arithmetic, *Advances in Computers* **1** (1960) 231–308.
- [31] Samueli, Henry, An improved search algorithm for the design of multiplierless FIR filters with powers-of-two coefficients, *Circuits and Systems, IEEE Transactions on* **36** no. 7 (1989) 1044–1047.
- [32] Selesnick, I. W., Lang, M., and Burrus, C. S., Constrained least square design for FIR filters without specified transition bands, *Signal Processing, IEEE Transactions on* **44** (1996) 1879–1892.
- [33] Shor, N. Z., Cut-off method with space extension in convex programming problems, *Cybernetics* **13** no. 1 (1977) 94–96.
- [34] Steiglitz, K., Parks, T. W., and Kaiser, J. F., METEOR: A constraint-based FIR filter design program, *Signal Processing, IEEE Transactions on* **40** (1992) 1901–1909.
- [35] Tuan, H. D., Son, T. T., Vo, B.-N., and Nguyen, T. Q., Efficient large-scale filter/filterbank design via LMI characterization of trigonometric curves, *Signal Processing, IEEE Transactions on* **55** (2007) 4394–4404.
- [36] Wu, Shao-Po, Boyd, Stephen, and Vandenberghe, Lieven, FIR filter design via spectral factorization and convex optimization, in *Applied and computational control, signals, and circuits*, Springer, pp. 215–245.
- [37] Xu, Dexiang J, and Daley, Michael L, Design of optimal digital filter using a parallel genetic algorithm, *Circuits and Systems II: Analog and Digital Signal Processing, IEEE Transactions on* **42** no. 10 (1995) 673–675.
- [38] Yudin, D. B., and Nemirovskii, A. S., Informational complexity and effective methods of solution for convex extremal problems, *Ekonomika i Matematicheskie Metody* **12** no. 2 (1976) 357–369.
- [39] Zhou, Xingbao, Luk, Wai-Shing, Zhou, Hai, Yang, Fan, Yan, Changhao, and Zeng, Xuan, Multi-parameter clock skew scheduling, *Integration, the VLSI Journal* **48** (2015) 129–137.